

会得到这张表。

.line: 原始 C 源程序中的行号和 .text 节中机器指令之间的映射。只有以 -g 选项调用编译器驱动程序时, 才会得到这张表。

.strtab: 一个字符串表, 其内容包括 .symtab 和 .debug 节中的符号表, 以及节头部中的节名字。字符串表就是以 null 结尾的字符串的序列。

旁注 为什么未初始化的数据称为 .bss

用术语 .bss 来表示未初始化的数据是很普遍的。它起始于 IBM 704 汇编语言(大约在 1957 年)中“块存储开始(Block Storage Start)”指令的首字母缩写, 并沿用至今。一种记住 .data 和 .bss 节之间区别的简单方法是把“bss”看成是“更好地节省空间(Better Save Space)”的缩写。

7.5 符号和符号表

每个可重定位目标模块 m 都有一个符号表, 它包含 m 定义和引用的符号的信息。在链接器的上下文中, 有三种不同的符号:

- 由模块 m 定义并能被其他模块引用的全局符号。全局链接器符号对应于非静态的 C 函数和全局变量。
- 由其他模块定义并被模块 m 引用的全局符号。这些符号称为外部符号, 对应于在其他模块中定义的非静态 C 函数和全局变量。
- 只被模块 m 定义和引用的局部符号。它们对应于带 static 属性的 C 函数和全局变量。这些符号在模块 m 中任何位置都可见, 但是不能被其他模块引用。

认识到本地链接器符号和本地程序变量不同是很重要的。.symtab 中的符号表不包含对应于本地非静态程序变量的任何符号。这些符号在运行时在栈中被管理, 链接器对此类符号不感兴趣。

有趣的是, 定义为带有 C static 属性的本地过程变量是不在栈中管理的。相反, 编译器在 .data 或 .bss 中为每个定义分配空间, 并在符号表中创建一个有唯一名字的本地链接器符号。比如, 假设在同一模块中的两个函数各自定义了一个静态局部变量 x :

```

1  int f()
2  {
3      static int x = 0;
4      return x;
5  }
6
7  int g()
8  {
9      static int x = 1;
10     return x;
11 }
```

在这种情况下, 编译器向汇编器输出两个不同名字的局部链接器符号。比如, 它可以用 $x.1$ 表示函数 f 中的定义, 而用 $x.2$ 表示函数 g 中的定义。

给 C 语言初学者 利用 static 属性隐藏变量和函数名字

C 程序员使用 static 属性隐藏模块内部的变量和函数声明, 就像你在 Java 和 C++

中使用 `public` 和 `private` 声明一样。在 C 中，源文件扮演模块的角色。任何带有 `static` 属性声明的全局变量或者函数都是模块私有的。类似地，任何不带 `static` 属性声明的全局变量和函数都是公共的，可以被其他模块访问。尽可能用 `static` 属性来保护你的变量和函数是很好的编程习惯。

符号表是由汇编器构造的，使用编译器输出到汇编语言 `.s` 文件中的符号。`.symtab` 节中包含 ELF 符号表。这张符号表包含一个条目的数组。图 7-4 展示了每个条目的格式。

```

code/link/elfstructs.c
1  typedef struct {
2      int    name;        /* String table offset */
3      char  type:4,       /* Function or data (4 bits) */
4          binding:4;      /* Local or global (4 bits) */
5      char  reserved;     /* Unused */
6      short section;      /* Section header index */
7      long  value;        /* Section offset or absolute address */
8      long  size;         /* Object size in bytes */
9  } Elf64_Symbol;
code/link/elfstructs.c

```

图 7-4 ELF 符号表条目。type 和 binding 字段每个都是 4 位

`name` 是字符串表中的字节偏移，指向符号的以 `null` 结尾的字符串名字。`value` 是符号的地址。对于可重定位的模块来说，`value` 是距定义目标的节的起始位置的偏移。对于可执行目标文件来说，该值是一个绝对运行时地址。`size` 是目标的大小（以字节为单位）。`type` 通常要么是数据，要么是函数。符号表还可以包含各个节的条目，以及对对应原始源文件的路径名的条目。所以这些目标的类型也有所不同。`binding` 字段表示符号是本地的还是全局的。

每个符号都被分配到目标文件的某个节，由 `section` 字段表示，该字段也是一个到节头部表的索引。有三个特殊的伪节（pseudosection），它们在节头部表中是没有条目的：`ABS` 代表不该被重定位的符号；`UNDEF` 代表未定义的符号，也就是在本目标模块中引用，但是却在其他地方定义的符号；`COMMON` 表示还未被分配位置的未初始化的数据目标。对于 `COMMON` 符号，`value` 字段给出对齐要求，而 `size` 给出最小的大小。注意，只有可重定位目标文件中才有这些伪节，可执行目标文件中是没有的。

`COMMON` 和 `.bss` 的区别很细微。现代的 GCC 版本根据以下规则来将可重定位目标文件中的符号分配到 `COMMON` 和 `.bss` 中：

`COMMON` 未初始化的全局变量

`.bss` 未初始化的静态变量，以及初始化为 0 的全局或静态变量

采用这种看上去很绝对的区分方式的原因来自于链接器执行符号解析的方式，我们会在 7.6 节中加以解释。

GNU `READELF` 程序是一个查看目标文件内容的很方便的工具。比如，下面是图 7-1 中示例程序的可重定位目标文件 `main.o` 的符号表中的最后三个条目。开始的 8 个条目没有显示出来，它们是链接器内部使用的局部符号。

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
8:	0000000000000000	24	FUNC	GLOBAL	DEFAULT	1	main
9:	0000000000000000	8	OBJECT	GLOBAL	DEFAULT	3	array
10:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	sum

在这个例子中，我们看到全局符号 `main` 定义的条目，它是一个位于 `.text` 节中偏移量为 0(即 `value` 值)处的 24 字节函数。其后跟随着的是全局符号 `array` 的定义，它是一个位于 `.data` 节中偏移量为 0 处的 8 字节目标。最后一个条目来自对外部符号 `sum` 的引用。`READELF` 用一个整数索引来标识每个节。`Ndx=1` 表示 `.text` 节，而 `Ndx=3` 表示 `.data` 节。



练习题 7.1 这个题目针对图 7-5 中的 `m.o` 和 `swap.o` 模块。对于每个在 `swap.o` 中定义或引用的符号，请指出它是否在模块 `swap.o` 中的 `.symtab` 节中有一个符号表条目。如果是，请指出定义该符号的模块(`swap.o` 或者 `m.o`)、符号类型(局部、全局或者外部)以及它在模块中被分配到的节(`.text`、`.data`、`.bss` 或 `COMMON`)。

符号	.symtab 条目?	符号类型	在哪个模块中定义	节
<code>buf</code>				
<code>bufp0</code>				
<code>bufp1</code>				
<code>swap</code>				
<code>temp</code>				

code/link/m.c

```
1 void swap();
2
3 int buf[2] = {1, 2};
4
5 int main()
6 {
7     swap();
8     return 0;
9 }
```

a) m.c

code/link/m.c

code/link/swap.c

```
1 extern int buf[];
2
3 int *bufp0 = &buf[0];
4 int *bufp1;
5
6 void swap()
7 {
8     int temp;
9
10    bufp1 = &buf[1];
11    temp = *bufp0;
12    *bufp0 = *bufp1;
13    *bufp1 = temp;
14 }
```

b) swap.c

code/link/swap.c

图 7-5 练习题 7.1 的示例程序

7.6 符号解析

链接器解析符号引用的方法是将每个引用与它输入的可重定位目标文件的符号表中的一个确定的符号定义关联起来。对那些和引用定义在相同模块中的局部符号的引用，符号解析是非常简单明了的。编译器只允许每个模块中每个局部符号有一个定义。静态局部变量也会有本地链接器符号，编译器还要确保它们拥有唯一的名字。

不过，对全局符号的引用解析就棘手得多。当编译器遇到一个不是在当前模块中定义的符号(变量或函数名)时，会假设该符号是在其他某个模块中定义的，生成一个链接器符号表条目，并把它交给链接器处理。如果链接器在它的任何输入模块中都找不到这个被引用符号的定义，就输出一条(通常很难阅读的)错误信息并终止。比如，如果我们试着在一